

Compte-rendu 3 : Fonctions

Important :

- Pour limiter le risque d'erreur, pensez à renommer ce fichier sous la forme : CR3-GRP-QUADRINOME suivi de son extension .docx, .odt ou .pdf. *Par exemple, le quadrinome 17 du groupe C qui dépose un fichier PDF nommera son fichier CR3-C-17.pdf*
- Remplissez bien (à nouveau) le nom et numéro de groupe ci-dessous (ex. C-17) et surtout, le prénom et nom des étudiantes et étudiants composant le groupe.
- Seul UN seul des membres du quadrinome doit déposer le compte-rendu. Si plusieurs compte-rendu sont rendus par le même groupe, un seul sera noté (peut-être le plus mauvais...)

Nom ET numéro de groupe :	B-03
---------------------------	------

Prénom	Nom	Adresse e-mail	Pourcentage d'implication dans le travail réalisé*
Théo	Barbe	theo.barbe1@etu.cyu.fr	32,5%
Mael	Willigens	mael.willigens@etu.cyu.fr	25%
Nathan	Absil	nathan.absil@etu.cyu.fr	32,5%
Eric	Lambert	eric.lambert1@etu.cyu.fr	10%

* la somme des pourcentages de cette colonne doit faire 100 % (aux arrondis près)

Leçons

Capsule-vidéo n°3-a

- Titre de la vidéo : fonctions
- Ce qu'on a appris avec cette vidéo : intérêt des fonctions, définir et utiliser une fonction, les paramètres et valeur de retour d'une fonction
- Exemple de code python illustrant ce qu'on a appris avec cette vidéo :

```
def calculeMasseElement (masse, proportion):  
    return masse*(proportion/100)
```

Capsule-vidéo n°3-b

- Titre de la vidéo : Introduction à la programmation Python 3c les procédures
- Ce qu'on a appris avec cette vidéo : Qu'est ce qu'une procédure, comment on l'utilise et la créer (pas de return)
- Exemple de code python illustrant ce qu'on a appris avec cette vidéo :

```
def afficheCamembert(nom, proportion):
    chiffres = [proportion, 100-proportion]
    labels = [nom, "Autres"]
    plt.pie(chiffres, labels=labels, autopct="%1.1f%%")
    plt.title("Proportion dans l'organisme")
    plt.show()
```

Capsule-vidéo n°3-c

- Titre de la vidéo : portée des variables
- Ce qu'on a appris avec cette vidéo : Comment utiliser des variables associées à une fonction. Et comment identifier et comprendre comment les utiliser dans l'exécution d'une fonction
- Exemple de code python illustrant ce qu'on a appris avec cette vidéo :

```
def calculeMasseElement (masse, proportion):
    return masse*(proportion/100)
```

```
def f1 (a, b) :
```

```
    a = 10
```

```
    b = a - c
```

```
    return b
```

```
def f2 (a, b) :
```

```
    c = 3 + f1 (a, b)
```

```
    return c
```

```
a = 1
```

```
b = 2
```

```
c = 3
```

```
b = f1 (a, c)
```

```
c = f2 (b, c)
```

```
print (a, end=" ")
```

```
print (b, end=" ")
```

```
print (c, end=" ")
```

Capsule-vidéo n°3-d

- Titre de la vidéo : Bibliothèques

- Ce qu'on a appris avec cette vidéo : Les modules sont regroupés dans des bibliothèques il existe des dizaines de bibliothèques pour différentes utilisations jeux vidéos maths etc.... pour cela il faut utiliser `from` et `import` le nom du module et le nom l'outil qu'on veut utiliser
- Exemple de code python illustrant ce qu'on a appris avec cette vidéo :

```
import matplotlib.pyplot as plt
```

Capsule-vidéo n°3-e

- Titre de la vidéo : Écrire proprement son programme
- Ce qu'on a appris avec cette vidéo : il faut espacer mettre `#` pour répertorié et trier pour rendre le plus ranger et compréhensible

Exercice 2.1 : Visualisation de la composition chimique du corps humain

Énoncé

Exercice 3.1 : Visualisation de la composition chimique du corps humain (suite)

Dans cet exercice, vous allez modifier et compléter le programme de l'exercice précédent.

1. Reprenez le programme que vous avez écrit lors de l'exercice précédent, qui demandait à l'utilisateur la masse totale du corps et le rang de l'élément dont il voulait connaître la masse.
2. Modifiez votre programme pour répartir le code en fonctions et procédures de la manière suivante :
 - une fonction `calculeMasseElement` qui prend en paramètre une masse totale et une proportion d'élément (en pourcentage) et qui retourne (donc avec un `return`, et il ne doit pas y avoir de `print`) la masse calculée.
 - Une procédure `afficheMessageProportion` qui prend en paramètre une proportion d'élément (en pourcentage) et qui ne fait qu'afficher (avec `print`) les messages correspondants (« élément majoritaire », important, faible quantité, minoritaire,...) sans retourner de valeur (donc sans `return`)
 - une procédure `afficheCamembert` qui prend en paramètre un nom d'élément, la proportion de cet élément et qui affiche un camembert avec la proportion de l'élément choisi par rapport à la masse totale du corps. Lors de l'affichage du camembert, seul l'élément choisi par l'utilisateur et le reste des éléments doivent être représentés. L'utilisation de la représentation en camembert (**pie**) de la bibliothèque **matplotlib** est expliqué à la fin de l'exercice. Le camembert doit indiquer
 - Le nom de l'élément sélectionné.
 - La proportion de cet élément par rapport au reste du corps.

3. Dans le bloc principal, la liste *proportion* sera définie, puis il sera demandé à l'utilisateur la masse totale du corps et le rang de l'élément dont il veut connaître la masse et enfin, la fonction *calculeMasseElement* sera appelée et le résultat récupéré sera affiché, et les procédures *afficheMessageProportion* et *afficheCamembert* seront également appelées pour afficher le message (important, faible, ...) et le camembert.

Code

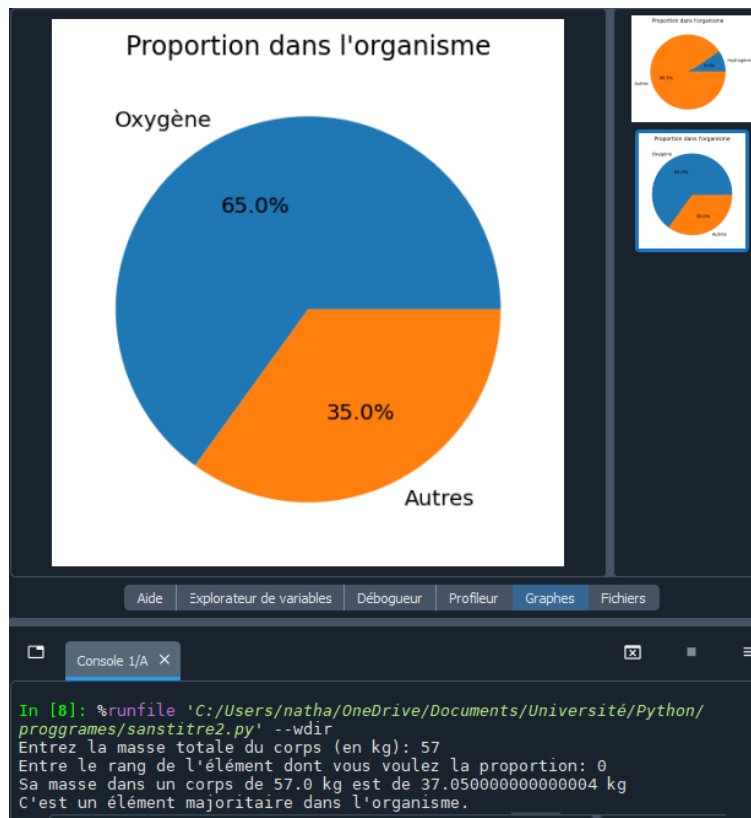
Copier/coller votre code dans le cadre ci-dessous

```
import matplotlib.pyplot as plt
# FONCTIONS ET PROCEDURES
#
# Fonction qui calcule masse de l'élément
def calculeMasseElement (masse, proportion):
    return masse*(proportion/100)
# Procédure qui affiche le message pour la proportion
def afficheMessageProportion(proportion):
    if proportion > 50:
        print ("C'est un élément majoritaire dans l'organisme.")
    elif proportion >10 and proportion <=50:
        print ("C'est un élément important dans l'organisme.")
    elif proportion >= 1 and proportion <=10:
        print("C'est un élément en faible quantité dans l'organisme.")
    else:
        print("C'est un élément minoritaire dans l'organisme.")
# Procédure pour le camembert
def afficheCamembert(nom, proportion):
    chiffres = [proportion, 100-proportion]
    labels = [nom, "Autres"]
    plt.pie(chiffres, labels=labels, autopct="%1.1f%%")
    plt.title("Proportion dans l'organisme")
    plt.show()
#####
# Bloc d'instructions principal
proportions = [['Oxygène', 'O', 65], ['Carbone', 'C', 18.5], ['Hydrogène', 'H', 9.5], ['Azote', 'N', 3.3], ['Calcium', 'Ca', 1.5], ['Phosphore', 'P', 1], ['Potassium', 'K', 0.4], ['Soufre', 'S', 0.3], ['Sodium', 'Na', 0.2], ['Chlore', 'Cl', 0.2], ['Magnésium', 'Mg', 0.1]]
masse= float (input ("Entrez la masse totale du corps (en kg): "))
rang= int (input("Entre le rang de l'élément dont vous voulez la proportion: "))
if rang > 11:
    print ("Rang inexistant ")
element= proportions [rang]
nom= element[0]
symbole= element[1]
proportion= element[2]
# Appel fonction calcul masse element
masse_calculée= calculeMasseElement(masse, proportion)
print ("Sa masse dans un corps de",masse, "kg est de",masse_calculée,"kg" )
# Appel procédure proportion
afficheMessageProportion(proportion)
# Appel procédure camembert
afficheCamembert(nom, proportion)
```

Résultat

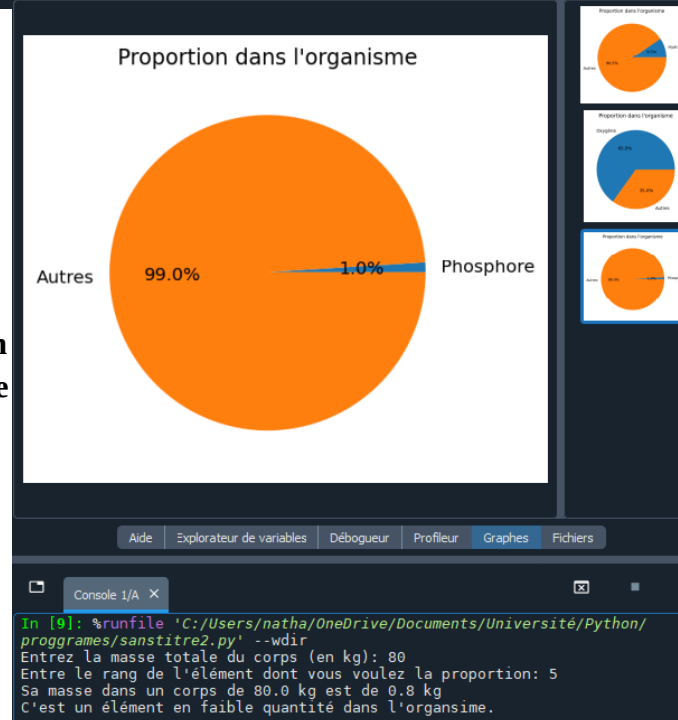
Test 1

Paramètre : masse totale et la proportion de l'élément chimique



Test 2

Paramètre : masse totale et la proportion de l'élément chimique



Réflexion

Import, def, return, input, if /else, print, variables

Exercice 2.2 : Calcul de la croissance d'une population bactérienne avec une limite environnementale

Énoncé

Exercice 3.2 : Visualisation de l'évolution d'une population bactérienne

Dans cet exercice, vous allez transformer le programme que vous avez écrit précédemment en une fonction générique. Vous aurez également à afficher la courbe représentant les résultats obtenus en fonction du temps d'observation.

1. **Reprenez votre code** de l'exercice précédent où vous avez modélisé la croissance bactérienne qui double à chaque cycle jusqu'à atteindre la capacité maximale `N_max` puis meurt suivant un taux de mortalité de `M` % à chaque cycle, **et transformez votre programme en une fonction que vous appellerez `calculeNbBacteries`**. Votre fonction doit prendre en paramètre :

- `N0` : le nombre initial de bactéries,
- `t_cycle` : le temps nécessaire pour un cycle en minutes,
- `N_max` : la capacité maximale de l'environnement (la population maximale possible),
- `M` : le taux de mortalité des bactéries en pourcentage une fois la capacité maximale atteinte,
- `temps_total` : le temps total d'observation en minutes.

La fonction devra calculer et retourner le nombre de bactéries lorsque le `temps_total` d'observation est atteint.

- Demandez à l'utilisateur de rentrer un nombre initial de bactéries, le temps d'un cycle, la capacité maximale de l'environnement et le taux de mortalité.
- Vous devrez ensuite faire varier `temps_total` en utilisant les valeurs suivantes : 0, 250, 500, 750, 1000, 1250 et 1500 minutes, et observer l'évolution de la population à chaque instant. Pour cela, vous appellerez votre fonction `calculeNbBacteries` depuis votre bloc d'instruction principal avec les 6 valeurs indiquées pour le `temps_total`.
- **Utilisation de `matplotlib.pyplot`** pour tracer la courbe représentant l'évolution de la population au cours du temps. L'axe des x sera le temps (`temps_total`) et l'axe des y représentera la taille de la population bactérienne.

Code

Copier/coller votre code dans le cadre ci-dessous

```
import matplotlib.pyplot as plt

# FONCTIONS ET PROCEDURES

#
```

```

def calculNbBacteries (bacteriesi, tempsdiv, N_max, M, t):

    nbr_cycles=(t//tempsdiv)

    N= bacteriesi*(2)**(t//tempsdiv)

    if N <= N_max:

        return N

    else:

        N_final=N_max*(1-M/100)**nbr_cycles

        return N_final

# Bloc d'instructions principal

bacteriesi= float (input("Entrez le nombre initial de bactéries"))

tempsdiv= float (input("Entrez le temps nécessaire pour une division (en minutes)"))

N_max= int (input("Entrez la capacité maximale de l'environnement"))

M= int (input("Entrez le taux de mortalité (en %)"))

liste_temps = [0, 250, 500, 750, 1000, 1250, 1500]

resultats_population= []

for t in liste_temps:

    population = calculNbBacteries(bacteriesi, tempsdiv, N_max, M, t)

    resultats_population.append(population)

#### Procédure pour le graphique

plt.plot(liste_temps, resultats_population, marker='o')

plt.title("Evolution de la population bactérienne au cours du temps")

plt.xlabel("Temps (en minutes)")

plt.ylabel("Population bactérienne")

```



**Résultat
Test 1**

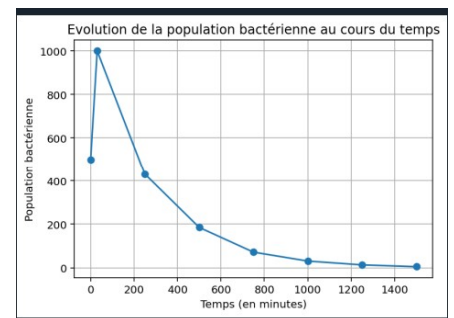
Paramètre : N_0 , tps d'un cycle, N_{max} , Le taux de mortalité, le tps total d'observation

Import, def, return, input, if /else, print, variables

Réflexion

Remarque : ici la liste donnée pour les abscisses a une échelle trop grande par rapport à un cycle. Le N_{max} étant de 1000 n'apparaît pas sur la courbe. Cependant si dans la liste on rajoute la valeur 30 cela nous affichera le N_{max} autrement dit une croissance puis une décroissance exponentielle(30 étant le nombre de minute nécessaire à un cycle et d'après la ligne de calcul :

$N = \text{bacteries}_i * (2)^{(t/\text{tempsdiv})}$ alors N_{max} est atteint au bout de 30 min et donne alors ce graphique :



Conclusion

Que devons-nous retenir d'important sur ce chapitre de cours ? Qu'y avons nous appris ?

Comment utiliser une fonction, la différence avec une procédure, comment structurer son programme et utiliser les bibliothèque notamment pour construire des graphs